

AOE/CS/ME 6444 Verification and Validation in Scientific Computing
Spring 2026
Instructor: Dr. Chris Roy

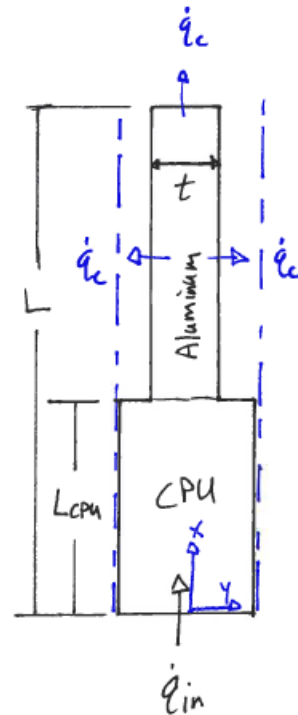
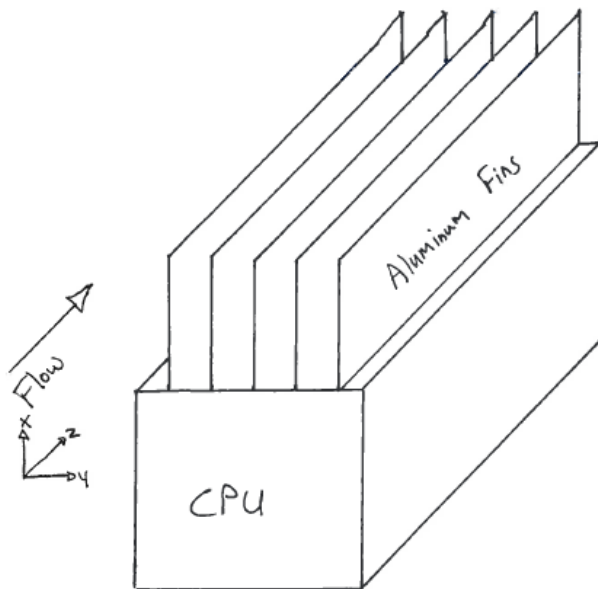
Homework #1
Due Tuesday February 17, 2026 by 11pm

Please upload your homework assignments in PDF format to the appropriate assignment section of Canvas. Please also use the following file naming convention: VVSC_Lastname_Firstname_HW1.pdf. If you have problems getting your homework into PDF format, then let me know.

For this homework, you will need to perform a nondeterministic analysis using a deterministic heat conduction MATLAB code that I will supply. Consider that a high-performance CPU has a Thermal Design Power (TDP) of 125 W. An aluminum heat sink with 5 fins sits on top of the CPU and air blows over this heat sink at 1 m/s to dissipate the heat as shown in the left figure. An approximate analysis has been conducted where the convective cooling coefficient has been averaged in the z-direction according to

$$\bar{h} = \frac{1}{W} \int_0^W h(z) dz = 15 \frac{W}{m^2 K}$$

and the convective cooling is then approximated using an average air temperature as $q_c = \bar{h}(T - T_\infty)$.



To simplify the analysis, only one aluminum fin and the corresponding portion of the CPU will be considered. The convective heat flux from the aluminum fin is found as specified above and there is assumed to be no heat flux in the y-direction from the CPU due to symmetry. The governing equation is thus

$$k_{Al} \frac{d^2 T}{dx^2} = \frac{2\bar{h}}{t} (T - \bar{T}_\infty)$$

with boundary conditions

$$\begin{aligned} k_{CPU} \frac{dT}{dx} \Big|_{x=0} &= - \frac{TDP}{W \cdot t \cdot N_{fins}} & x = 0 \\ k_{Al} \frac{dT}{dx} \Big|_{x=L} &= -\bar{h} [T(x=L) - \bar{T}_\infty] & x = L \end{aligned}$$

Considering the conditions given in the table below (these are the default values in the MATLAB code I will supply), estimate cumulative distribution functions of the base temperature $T(x=0)$ using 10, 100, and 1000 samples assuming 1) the thermal conductivity of the CPU obeys a normal distribution with a mean value of 0.40 W/(m·K) and a standard deviation of 0.07 W/(m·K) and 2) the fin thickness obeys a normal distribution with a mean value of 0.3 cm and a standard deviation of 0.05 cm. Since the 10 sample case will be strongly dependent on which samples you choose, you should repeat this case a few times to see how the answers change. Find the probability that the CPU base temperature will exceed the peak allowable temperature of 85 deg. C. Note that this base temperature is returned by the function *heatcondsolve* as the variable *Tbase*. The default numerical parameters should be used: 33 nodes and a relative iterative convergence tolerance of 1×10^{-8} . Examine both standard Monte Carlo sampling and Latin Hypercube sampling. You can use the MATLAB **randn()** function to sample from the normal (Gaussian) distribution using Monte Carlo and the **lhsnorm()** function to use Latin hypercube sampling. Note that the sigma value that the **lhsnorm()** function requests is actually the variance of the normal distribution (standard deviation squared). Submit your MATLAB *.m file as well as your results and a brief analysis interpreting the results (no more than a page of text please).

Table of Input Parameters

Thermal Design Power, <i>TDP</i>	125 W
Number of fins, <i>NumFins</i>	5
Total length (CPU + Fin), <i>L</i>	8 cm
Length of CPU, <i>L_{CPU}</i>	3 cm
Fin thickness, <i>thick</i>	N(0.3, 0.05) cm , COV = 16.7%
Thermal conductivity of Aluminum, <i>k_{Metal}</i>	160 W/(m·K)
Thermal conductivity of CPU, <i>k_{CPU}</i>	N(0.4, 0.07) W/(m·K), COV = 17.5%
Averaged convection coefficient, <i>hbar</i>	15 W/(m ² K)
Averaged air temperature, <i>Tbar</i>	330 K

Some MATLAB commands you may find useful include:

```
cdfplot() #for plotting the empirical CDF
ecdf() with stairs() #for plotting the empirical CDF
normrnd(mu, sigma) #generates a random number from the
                    normal distribution with mean parameter mu and
                    standard deviation parameter sigma (e.g., for
                    Monte Carlo sampling)
lhsnorm(mu, sigma, n) #sample from the normal distribution
                      using Latin hypercube sampling
```

Note: for lhsnorm(mu, sigma, n):

```
sigma = [sigma_1^2, 0; 0, sigma_2^2]
mu = [mu_1; mu_2]
n = sample size
```

The sigma here is a covariance matrix, where the diagonal entries should be the variances of your two uncertain inputs (variance is the standard deviation squared). Since the off-diagonal terms are zero, this assumes no correlation between these two uncertainties. The use of the name “sigma” for the argument here by MATLAB is confusing.